

Software Development Engineer in Test

Suppose you are part of a scrum team developing data warehouse for Bloomberg to analyze FX deals. One of the customer stories is to accept deals details from and persist them into DB.

Request logic as following:

- Request Fields (Deal Unique Id, From Currency ISO Code "Ordering Currency" To Currency ISO Code, Deal timestamp, Deal Amount in ordering currency).
- Validate row structure. (e.g.: Missing fields, Type format...etc. We do not expect you to cover all possible cases, but we'll look to how you'll implement validations)
- The system should not import the same request twice.
- No rollback allowed; every row imported should be saved in DB.

Deliverables should be ready to work including:

- Use Actual Db, you can select between (Postgres, MySQL or MongoDB)
- Workable deployment including sample file (Docker Compose).
- Maven or Gradle project is required for full source code.
- Proper error/exception handling.
- Proper Logging.
- Proper unit testing/Integration Tests (if any)/Spring Mvc Tests with respected Coverage.
- Proper documentation using md.
- Use K6 to Test Performance
- Using Postman to test API, postman collection should be submitted

- Delivered over Github.com.
- Makefile to streamline running application (plus).

Additional Testing Requirements

Coverage & Build Gate

Coverage must be checked using JaCoCo and targeting 100% for production code related to parsing, validation, deduplication, and import flow. The build should fail if the target is not met. If you exclude trivial/bootstrap/generated code, list and justify the exclusions in README.md.

Unit Tests

Provide unit tests covering core business logic such that the coverage target above is achieved. Include negative and boundary conditions you deem necessary; you decide the scope and structure.

Integration Tests (if needed)

Where you judge it necessary to validate behavior at the database boundary, provide integration tests that run against a real DB consistent with your stack. Integration tests should contribute to the same JaCoCo report. Document what is covered by unit vs. integration.

API Tests with RestAssured

Provide a comprehensive RESTAssured test suite against your public API that demonstrates - through runnable tests and assertions - that the implementation satisfies the assignment requirements (field acceptance, row level validation behavior, prevention of importing the same request twice, and partial success semantics with no rollback). Do not limit yourself to a small representative set; include success, error, and edge conditions you consider necessary. In README.md include a brief requirement to tests mapping and commands to run the suite and view coverage.

Reproducibility

All tests must run from a clean checkout via the Makefile and/or build tool (e.g., make up, make test, make coverage) without manual steps.